

Article

Low-FPS Multi-Object Multi-Camera Tracking via Deep Learning

Yu-Heng Hsieh ^{*}, Chen-Chun Kao, Chiung-Han Lai, Keng-Pei Lin, Shun-Yu Yang and Shyan-Ming Yuan ^{*}

Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu 30010, Taiwan; b311551147.cs11@nycu.edu.tw (C.-C.K.); jonaslaisad.cs12@nycu.edu.tw (C.-H.L.); jonathan03020208.cs12@nycu.edu.tw (K.-P.L.); aebandafg.cs12@nycu.edu.tw (S.-Y.Y.)
* Correspondence: k28998989.cs11@nycu.edu.tw (Y.-H.H.); smyuan@nycu.edu.tw (S.-M.Y.)

Abstract: Multi-Object Multi-Camera Tracking (MOMCT) is essential for real-world applications but remains challenging in low-FPS environments where motion-based tracking is ineffective. This study proposes a novel vehicle tracking approach that integrates YOLOv9 for object detection, SwinReID for feature extraction, and a KNN-based matching algorithm to enhance tracking across multiple cameras. Our research leverages the AICUP 2024 Spring Competition dataset, which includes surveillance footage from Chiayi City, along with ImageNet for model training. To improve the accuracy, we incorporate scene segmentation to reduce false detections and optimize the buffer size per camera through an exhaustive search, adapting to varying perspectives and frame rates. Our method achieves a total score of 1.263626 based on IDF1 and MOTA metrics, demonstrating its effectiveness in low-FPS settings. These results contribute to the development of more robust MOMCT systems for traffic monitoring and surveillance.

Keywords: MOMCT; YoLo v9; Swin Transformer; low FPS; deep learning



Academic Editors: George A. Papakostas, Jie Guo, Jing Hu and Ying Ouyang

Received: 1 February 2025

Revised: 19 March 2025

Accepted: 27 March 2025

Published: 29 March 2025

Citation: Hsieh, Y.-H.; Kao, C.-C.; Lai, C.-H.; Lin, K.-P.; Yang, S.-Y.; Yuan, S.-M. Low-FPS Multi-Object Multi-Camera Tracking via Deep Learning. *Electronics* **2025**, *14*, 1373. <https://doi.org/10.3390/electronics14071373>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Multi-Camera Multi-Object Tracking (MCMOT) is a key research area in deep learning, focusing on detecting and tracking multiple objects across video frames. Object detection involves two steps: localization, which identifies objects' positions using tightly bound boxes, and classification, which assigns categories and labels to the localized objects [1,2]. This can be achieved through two-stage methods, separating detection into localization and classification, or one-stage methods, which treat detection as a combined task [3].

Object tracking follows detection by assigning unique IDs to detected objects and tracking them across frames. Multi-Object Tracking (MOT) can be carried out through detection-based tracking or end-to-end deep neural networks. However, MOT faces challenges such as ID switching, where objects are assigned inconsistent IDs across frames, and occlusion, where objects are obscured by others. Despite advances, overcoming these challenges remains crucial for improving the accuracy and robustness of MOT systems.

Many MCMOT systems rely on high frame rates (FPS) to achieve accurate classification, often using motion detection to determine which camera an object belongs to. However, this approach is highly dependent on high FPS, as objects may easily disappear or reappear between frames. In some environments, such as underdeveloped regions or factory settings, cameras typically operate at lower FPS, making this a significant challenge.

This study applies Multi-Camera Multi-Object Tracking (MCMOT) to the problem of vehicle tracking, introducing several key innovations:

1. High-accuracy vehicle tracking in a low-FPS environment

Unlike most existing MCMOT studies [4–14], which incorporate motion detection and cross-road zone constraints to improve the tracking accuracy, this study focuses on an environment where the frame interval exceeds three seconds, rendering motion-based techniques ineffective. Instead, we employ YOLOv9 [15] for object detection and SwinReID for vehicle re-identification (ReID), achieving high IDF1 and MOTA scores despite the challenging conditions. Details of the IDF1 and MOTA scores can be found in Section 4.

2. Scene segmentation to filter out irrelevant detections

To address the misclassification of non-vehicle objects as vehicles by YOLOv9, we segment each camera's field of view into vehicle-permissible and non-permissible regions. By filtering out detections in restricted areas, we significantly improve the detection accuracy and prevent false positives from affecting the tracking results. Details of this approach can be found in Section 3.

3. Adaptive buffer size optimization for each camera

Traditional MCMOT methods typically track objects using only the bounding boxes or appearance features from the previous frame. However, since our dataset operates at nearly 1 FPS, relying solely on single-frame associations would significantly degrade the tracking performance. Furthermore, variations in camera angles and image clarity make a uniform buffer size across all cameras suboptimal. To address this, we perform an exhaustive search on the training set to determine the optimal buffer size for each camera, maximizing IDF1 and MOTA scores. Details of the adaptive buffer strategy are provided in Section 3.

This paper is organized as follows: Section 2 reviews related work on MCMOT, Section 3 presents the materials and methods, Section 4 details the experimental settings and results, and Section 5 concludes this paper with a discussion on future work.

2. Related Work

The rapidly developing deep learning techniques have an excellent performance in object detection, providing powerful tools for MCMOT [4–7]. Objects to track can be, for example, pedestrians on the street [16,17], vehicles in the road [8–14], animals [18,19], or sport players on the court [20,21].

In this section, we present three subsections discussing deep learning techniques and object tracking: (1) deep learning techniques and object tracking, (2) tools, models, and vehicle identification, and (3) computational cost challenges.

2.1. Deep Learning Techniques and Object Tracking

Deep learning has significantly advanced object tracking by improving the detection accuracy and robustness. Modern multi-object tracking (MOT) frameworks integrate object detection models like YOLO [15,22] with tracking algorithms such as SORT and Deep SORT [23] to enhance the tracking stability. These approaches address challenges such as occlusion, identity switching, and real-time processing, making them essential for applications like vehicle tracking and traffic monitoring.

The authors Lu et al. [11], in their article, proposed a real-time vehicle counting system for traffic analysis at busy intersections. The framework employs PP-YOLO for detection and Deep SORT for multi-object tracking. A shape-driven motion allocation strategy distinguishes vehicle trajectories, while spatial constraints minimize errors. To address occlusion and rapid velocity changes, detection augmentation techniques such as re-matching and single-object tracking are applied, along with a smooth Mahalanobis distance approach. Additionally, spatial tracking constraints filter out vehicles beyond the region of interest. The experimental results demonstrate high efficiency, achieving an S1-score of 0.9467, highlighting the system's robustness and accuracy.

The authors Cheng et al. [14], in their article, developed DATrack, a multi-class multi-object tracking (MCMOT) algorithm for self-driving vehicles, addressing challenges such as occlusion and intense camera movement. The algorithm employs TSCODE to decouple detection tasks, enhancing the classification and localization accuracy. An adaptive motion prediction (AMP) module improves the trajectory estimation by dynamically adjusting the Kalman filter based on the matching cost and detection confidence. Additionally, AMP mitigates the effects of the camera motion, refining the data association and tracking precision. DATrack surpasses leading methods on the BDD100K MOT dataset [24], achieving a MOTA of 58.6% and an IDF1 of 63.5%, while running in real time at 27.6 FPS on an NVIDIA RTX 2080Ti, NVIDIA, Santa Clara, CA, USA.

2.2. Tools, Models, and Vehicle Identification

Re-identification (ReID) [25–27] plays a crucial role in multi-object tracking (MOT), especially in scenarios with occlusions, viewpoint variations, and high visual similarity between objects. ReID utilizes deep learning models to extract feature vectors of pedestrians or vehicles, combined with metric learning techniques to enhance the matching accuracy. This enables tracking systems to associate the same target over time, even if it temporarily disappears from the frame or moves to a different viewpoint.

The authors Tran et al. [12], in their article, propose a spatial trajectory alignment approach for vehicle identification and counting in intelligent traffic systems. This method enhances the counting accuracy by integrating region of interest (ROI) detection with predefined movement tracks. YOLOv5 is used for vehicle detection, while SORT tracking links detected vehicles across frames to maintain consistent trajectories. By associating each vehicle's path with a specific movement, the approach improves precision at intersections with overlapping lanes. The system achieves 120 FPS on an NVIDIA Quadro RTX 8000 (NVIDIA, Santa Clara, CA, USA) and 20 FPS on a Jetson Xavier AGX (NVIDIA, Santa Clara, CA, USA), demonstrating its efficiency and scalability for real-time embedded applications.

The authors V.-H. Tran et al. [13], in their article, propose a robust vehicle counting system for crowded intersections, leveraging Scaled-YOLOv4 for detection and Deep SORT for multi-object tracking. The system improves accuracy through high-recall detection and effective feature matching, incorporating a cosine similarity-based scheme for movement prediction. The experimental results confirm its reliability in handling occlusions and environmental challenges while maintaining the real-time performance.

2.3. Computational Cost Challenges

Real-time vehicle tracking and counting systems must balance accuracy with computational efficiency, especially in resource-constrained environments. High-performance models often demand significant processing power, making them impractical for large-scale deployment or edge computing applications. To address this, researchers have developed methods that optimize model architectures, reduce redundant computations, and leverage efficient processing techniques. The following studies explore strategies to enhance the tracking performance while minimizing computational costs, enabling real-time traffic monitoring in various settings.

The authors Ha et al. [8], in their article, propose a detection–tracking–counting (DTC) framework for IoT edge computing, utilizing a lightweight YOLOv4-Tiny detector for vehicle identification, a Kalman Filter-based tracker with IoU and Mahalanobis distance metrics for robust multi-vehicle tracking, and a trajectory-based counting module to estimate the motion direction and exit time. Optimized for computational efficiency, the framework leverages parallel processing and CUDA acceleration for real-time execution on resource-constrained devices. The experimental results demonstrate superior accuracy

and speed compared to existing methods, enabling large-scale deployment without costly infrastructure upgrades.

The authors Gloudemans et al. [9], in their article, propose LBT-Count, an edge-computing-based method for efficient vehicle movement counting at intersections. By processing only selected image regions based on predicted object locations, LBT-Count reduces computational costs while maintaining accuracy. Unlike traditional detect-track-count approaches, it employs a localizer to detect objects within cropped segments, eliminating the need for full-frame detection. The experimental results show that LBT-Count is 52% faster than conventional tracking-by-detection frameworks while delivering a competitive performance, making it a scalable and cost-effective solution for real-time urban traffic monitoring.

The authors Kocur et al. [10], in their article, propose a real-time traffic flow analysis approach using CenterTrack, a keypoint-based object detection and tracking network built on CenterNet. By estimating object displacement between frames and integrating an IoU-based tracking algorithm, the method enhances the tracking efficiency. Vehicle movements are classified using 2D field modeling, with categories refined through non-maximum suppression. Additionally, linear regression is applied to estimate vehicle exit times, demonstrating a lightweight and effective solution for real-time traffic monitoring in resource-constrained environments.

3. Materials and Methods

In the part related to the research method and its application, we employ analytical processes and numerical data through dataset processing. Specifically, we analyze vehicle tracking performance using deep learning-based detection and re-identification models in a low-FPS environment. This study integrates YOLOv9 for object detection, SwinReID for feature extraction, and a KNN-based matching algorithm to track vehicles across multiple cameras. To ensure robustness, we optimize parameters such as buffer size and re-identification thresholds through exhaustive search.

To increase prediction accuracy, motion detection can be effective in high-FPS environments [8–14] due to the continuous movement of vehicles. However, in low-FPS environments, vehicles may not exhibit continuous movement, as demonstrated in Figure 1. In Figure 1a,b, captured by the same camera at frame t and frame $t + 1$, the silver car appears fully in frame t but only partially in frame $t + 1$. Since motion detection relies on continuous vehicle movement to enhance accuracy, it cannot be effectively used in this environment.

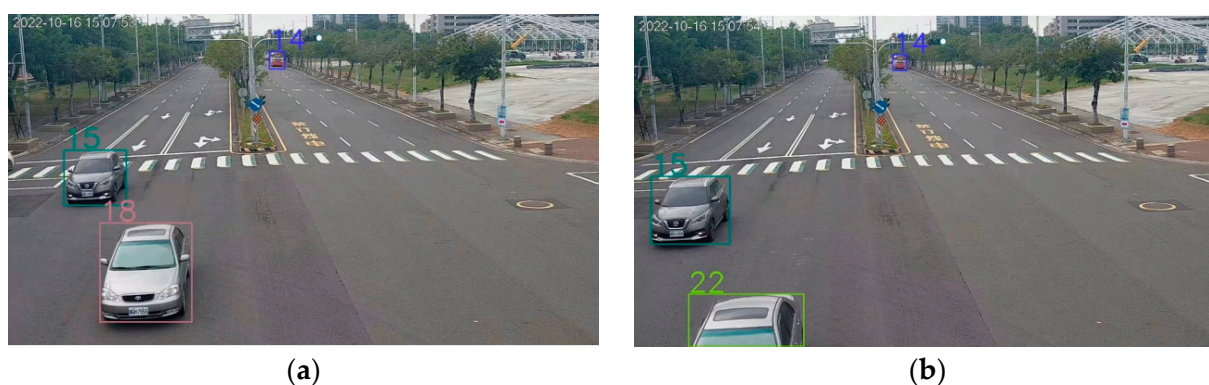


Figure 1. The two consecutive frames captured by the same camera. (a) Frame at time t , (b) frame at time $t + 1$.

This study aims to answer the following research questions:

- (1) How effective is our approach in tracking vehicles across multiple low-FPS cameras?

- (2) How does buffer size optimization impact tracking performance?
- (3) What is the effect of re-identification techniques on IDF1 and MOTA scores?

To evaluate our method, we conducted 5 sets of experiments under different parameter settings, including variations in buffer size, detection models, and ReID strategies. The results were validated using IDF1 and MOTA metrics.

3.1. Datasets

To construct our dataset, we collected video footage from eight surveillance cameras installed at key intersections in Chiayi City. The dataset development process follows a structured pipeline, as shown in Figure 2, which includes the following steps:

1. Capturing street-view footage from eight cameras in Chiayi City positioned at key traffic intersections and keeping it as object detection model datasets.
2. Manually annotating vehicle bounding boxes to create ground truth labels for detection and tracking.
3. Extracting individual vehicle images from the bounding box annotations in step 2 to build feature extraction datasets.

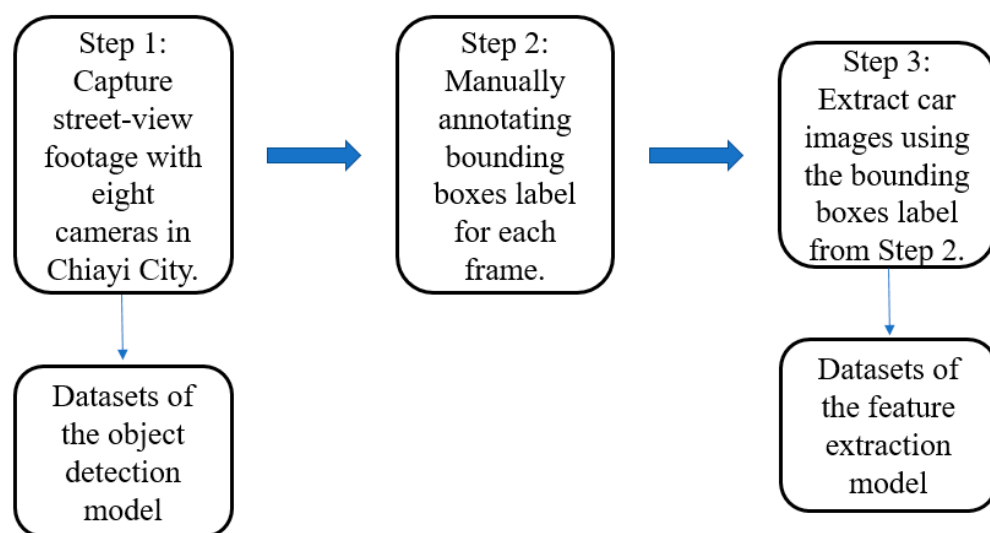


Figure 2. The dataset construction pipeline, following the AICUP competition guidelines. Street-view images captured by eight cameras in Chiayi City are used to generate labeled datasets for object detection and feature extraction.

Following this pipeline, the final dataset consists of 18,000 images from eight cameras, covering 5439 unique vehicle IDs. This dataset serves as the foundation for training both our object detection and re-identification models.

Unlike conventional datasets that rely on high-frame-rate sequences for motion-based tracking, our dataset is specifically designed for low-FPS environments by capturing frames at 1 FPS. The dataset enables robust tracking by integrating YOLOv9 for object detection, SwinReID for feature extraction, and a KNN-based matching algorithm to track vehicles across multiple cameras.

After dataset construction, we applied a series of preprocessing techniques to enhance the model robustness, which will be detailed in the next section.

3.1.1. Datasets of Object Detection Model

The datasets used in this paper are from the AICUP 2024 Spring Competition datasets [28], which have a frame rate of less than 1 FPS, making motion-based analysis challenging. The dataset consists of 18,000 images captured by eight cameras, covering

5439 unique vehicle IDs. Table 1 provides a comprehensive overview of the training and validation sets, which contain 15,000 and 3000 images, respectively, along with data distribution, recording dates, and times for each camera.

Table 1. Training and validation sets in our dataset.

Training Sets	Validation Sets
0902_150000_151900	1015_150000_151900
0902_190000_191900	1015_190000_191900
0903_150000_151900	
0903_190000_191900	
0924_150000_151900	
0924_190000_191900	
0925_150000_151900	
0925_190000_191900	
1016_150000_151900	
1016_190000_191900	

3.1.2. Datasets of ReID Model

During the ReID training process, the dataset is sourced from:

- The ground truth of the training set in Table 1 (including bounding boxes and ID labels).
- Data cropped by YOLOv9 and labeled with vehicle IDs.

For the second data source, we first use YOLOv9 to detect vehicles in an image and then calculate the Intersection over Union (IoU) between the detected bounding boxes and the ground truth bounding boxes. If the IoU exceeds 0.6, we assign the ID label from the ground truth bounding box to the YOLOv9-detected bounding box and include it in the training dataset.

3.2. Data Preprocessing

The datasets are annotated from real footage captured by street cameras in Chiayi City. Frames are extracted at 1 FPS, and any missing frames are due to the original recording conditions.

3.2.1. Data Preprocessing of Object Detection Datasets

We resize each original frame to a fixed resolution of (640, 640) to ensure consistency in input dimensions which helps stabilize model training. To enhance dataset diversity and improve the model's robustness, we apply Mosaic data augmentation, which combines four randomly selected training images into a single image. This technique effectively simulates various scene compositions, providing the model with a wider range of contextual variations.

Additionally, we incorporate left-right flipping with a 50% probability, allowing the model to learn symmetry-related patterns and improve its ability to recognize objects from different orientations. To further enhance generalization, we adjust the color properties of the images by modifying the hue (0.015), saturation (0.7), and brightness (0.4). These adjustments introduce natural variations in color and lighting conditions, helping the model perform more reliably under diverse environmental settings.

3.2.2. Data Preprocessing of Re-Identification Datasets

The input to SwinReID consists of cropped vehicle images. To prepare the images for the model, we first normalize the pixel values to a range between 0 and 1, ensuring consistency in input scaling. Next, we resize all images to a fixed resolution of (224, 224)

to standardize the input dimensions. To further refine the input distribution, we apply standardization using the mean and standard deviation values from ImageNet [29], which aids in stabilizing training and accelerating convergence.

During the data augmentation phase, we introduce various transformations to improve the model's capacity to adapt to diverse scenarios. We apply random horizontal flipping to expose the model to mirrored vehicle appearances, improving recognition under diverse orientations. Additionally, we use random padding followed by cropping to maintain the final image size at (224, 224) while introducing slight variations in object positioning. To further increase robustness, we incorporate Random Erasing [30], where randomly selected regions of the image are removed and replaced with the ImageNet mean values. This technique helps the model become more resistant to occlusions and partial visibility of objects. Each of these augmentation techniques is applied with a probability of 0.5 to balance diversity while preventing excessive alterations to the original images.

A second-layer pairing buffer further improves accuracy by segmenting previously captured vehicle images into upper, left, and right portions. This helps match vehicles that exit the frame. Only edge-detected vehicles enter this buffer to ensure relevance.

Experiments show this method improves tracking, especially for vehicles leaving the frame. However, some challenges remain, such as misassignments when new vehicles appear at frame edges and are stored in the buffer.

4. Experiments and Results

In this section, we introduce the model, parameter settings used in this paper, and our experimental results.

We utilized three servers to support different tasks. For re-identification (ReID) model training and inference, we employed a server equipped with an NVIDIA GeForce RTX 4070 GPU (12 GB), 13th Gen Intel® Core™ i7-13700 CPU, 32 GB RAM, and Ubuntu 22.04.4 LTS operating system. The YOLOv9 model training and inference were conducted on a server with an NVIDIA GeForce RTX 2080 GPU (8 GB), AMD Ryzen™ 5 3600 CPU, 48 GB RAM, and Windows 11 Education Edition operating system. Additionally, single-camera and multi-camera tracking tasks were performed on a server featuring an NVIDIA GeForce RTX 4090 GPU (24 GB), 12th Gen Intel® Core™ i7-12700K CPU, 64GB RAM, and Ubuntu 24.04 LTS operating system.

4.1. Experiments Model

In this subsection, we will talk about the object detection model and ReID model we used in this research.

4.1.1. Object Detection Model

We use YOLO v9 [15] as our object detection model to identify objects in each frame. For every processed frame, we extract the bounding boxes and assign unique IDs to the detected objects. These detection results are then stored in a buffer separately for each camera. These buffered data are later utilized for tracking and pairing objects across frames, ensuring the efficient handling of object movement and association in a low-FPS environment.

4.1.2. SwinReID Model

After obtaining the bounding boxes generated by YOLO v9, we proceed with the ReID process. For this, we employ a Swin Transformer-based model [31], referred to as SwinReID, whose architecture is illustrated in Figure 3. SwinReID utilizes the Swin-B Transformer model as the feature extractor, processing vehicle images to generate feature

representations. These representations are refined through a Batch Normalization (BN) layer before passing through a Fully Connected (FC) layer for classification.

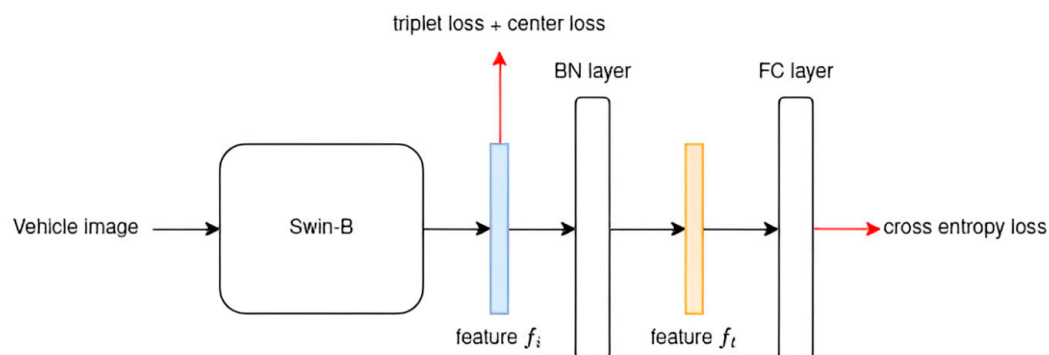


Figure 3. SwinReID architecture.

The primary function of SwinReID is to project vehicle images into the embedding space while ensuring two key properties:

- Images of the same vehicle should be mapped to proximate points in the embedding space.
- Images of different vehicles should be mapped to separate points in the embedding space.

By preserving these properties, determining whether two vehicle images belong to the same vehicle becomes straightforward. The embeddings of both images are obtained using SwinReID, and their distance is computed to assess similarity.

SwinReID is inspired by Swin Transformer [31] and TransReID [32]. We feed the vehicle images, cropped by YOLOv9, into the model. These images are then mapped into vectors f_i within the R^{2048} space by the Swin Transformer. Afterward, batch normalization [4] is applied to convert f_i into a vector f_t in the same R^{2048} space. During inference, we use f_t to compute the distance between vehicles.

We utilize triplet loss [33], center loss [34], and cross-entropy loss as the loss functions. The total loss is computed as follows:

$$total\ loss = triplet\ loss + center\ loss + cross\ entropy\ loss$$

where the margin for the triplet loss is set to 0.6. The model parameters are optimized using the AdamW [35] optimizer, with the weight decay coefficient set to 10^{-6} . We train the model for a total of 30 epochs, with an initial learning rate of 10^{-4} and a batch size of 10.

Before training with our datasets, we train the SwinReID with veri-776 [36] datasets.

4.2. Experiment Setting

In this subsection, we will talk about the parameter settings and training process of the two models.

4.2.1. Object Detection Model Training Settings

During the training of the YOLOv9 model, we fine-tuned the pre-trained weights from the COCO dataset, specifically using the yolov9-c.pt checkpoint. The learning rate was set to 0.0001, and all input images were resized to (640, 640) to maintain consistency. The model was trained for a total of five epochs.

For the loss calculation, we considered multiple factors to optimize the detection accuracy. The Bounding Box Loss was computed using the IoU (Intersection over Union) method to evaluate the accuracy of the predicted bounding boxes. Meanwhile, the Objectness Loss, which determines whether an object is present in a given region, was calculated using the

cross-entropy loss. To optimize the model's parameters, we employed the AdamW [35] optimizer, which enhances weight updates by incorporating decoupled weight decay.

Throughout the training process, we used mean Average Precision (mAP) as the primary evaluation metric to assess the model's performance in object detection. The mAP metric provides a comprehensive measure of both precision and recall, ensuring that the model accurately detects objects while minimizing false positives and false negatives.

4.2.2. SwinReID Training Settings

In the training process of SwinReID, each input consists of three vehicle images: anchor, positive, and negative. The anchor and positive images are different pictures of the same vehicle, while the negative image contains a vehicle different from the anchor and positive images. After image preprocessing and data augmentation, these three images are used as the input to SwinReID.

For both the VeRi-776 pre-training and competition dataset training, the same training process and hyperparameters are used. The only difference is that when sampling vehicle images from the competition dataset, if the original image's width and height are both greater than 128, there is a 0.1 probability that the image will undergo cropping. The image will be cropped to one of the following parts: top-left, top-right, upper half, left half, or right half, and then resized to (224, 224). This cropping technique was introduced because we found that vehicles at the edge of the camera's view may only show partial vehicle appearances. By using this technique, we simulate the appearance of vehicles at the edge of the camera's view.

For different cameras, we apply varying thresholds to determine whether two detected vehicles belong to the same vehicle. Given the influence of camera angles, lighting conditions, and resolution on the vehicle appearance, setting an appropriate threshold for each camera is crucial for accurate identification. To optimize this parameter, we conduct an exhaustive search to identify the threshold values that maximize the IDF1 and MOTA scores, systematically evaluating different settings to enhance the tracking accuracy and minimize misidentifications.

Additionally, buffer size plays a key role in vehicle tracking by allowing the system to compare frames from earlier timestamps. A small buffer size may cause vehicles that were occluded for too long to be mistakenly identified as new ones, while an excessively large buffer size could lead to incorrect associations between similar-looking vehicles. To assess its impact, we evaluate buffer sizes ranging from 1 to 15 and analyze their effect on the IDF1 and MOTA scores. As illustrated in Figure 4, the total score increases sharply with the buffer size, reaching an optimal range around five before gradually declining as the buffer size grows too large. This trend underscores the need for the careful selection of the buffer size to maintain a balance between accurate vehicle matching and avoiding misidentifications.

Table 2 presents both the optimized threshold values and corresponding buffer sizes for each of the eight cameras.

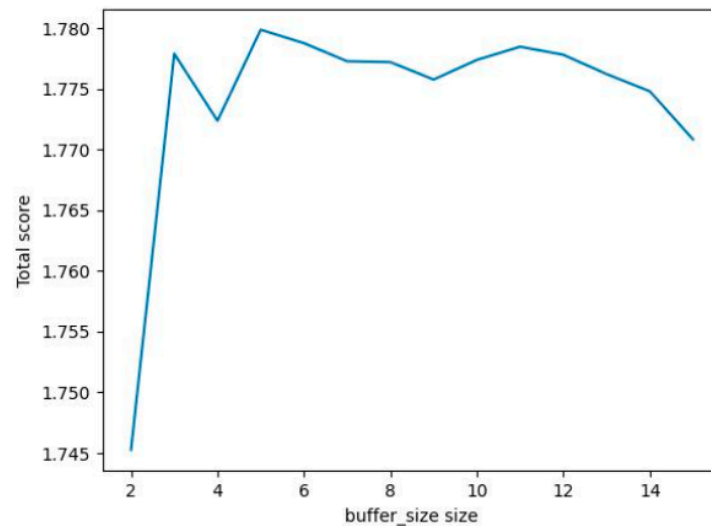


Figure 4. Relationship between buffer size and total score in camera 0.

Table 2. The threshold in 8 cameras.

Camera	Thresholds	Buffer Size
Camera 0	0.66	5
Camera 1	0.66	2
Camera 2	0.62	3
Camera 3	0.6	6
Camera 4	0.56	6
Camera 5	0.64	10
Camera 6	0.64	4
Camera 7	0.64	3

4.3. Our Method for MCMOT

In this subsection, we discuss the details of object pairing in a low-FPS environment, which consists of two steps: a single-camera tracking process and a cross-camera tracking process.

4.3.1. Single-Camera Tracking Process

The pseudo code of a single-camera process is shown as Algorithm 1. It begins by initializing an empty buffer, setting the maximum buffer size, and defining a similarity threshold (lines 1–3). The system then continuously processes incoming frames (line 5), extracting the features of all detected vehicles (line 6).

If the buffer contains previous vehicle data (line 8), all vehicle IDs in the new frame are reset to zero (line 9), and a similarity matrix is computed between the stored features and the newly detected vehicles (line 10). The algorithm then attempts to match each new vehicle to a buffered vehicle with the highest similarity score (lines 12–14). If the maximum similarity exceeds the threshold (line 15), the new vehicle is assigned the corresponding buffered vehicle's ID, and the similarity matrix entry for this pair is set to negative infinity to prevent duplicate matches (lines 16–17). If no sufficiently similar match is found, the new vehicle is assigned a unique ID (lines 18–19).

If the buffer is empty (line 22), all detected vehicles in the frame are assigned new IDs (line 23). The algorithm then updates the buffer by storing the extracted vehicle information, including features and IDs (line 26). To manage memory constraints, if the buffer exceeds

its maximum size, the oldest records are removed until the buffer size is within the limit (lines 28–30).

This process repeats as long as new frames are received (line 31), ensuring efficient vehicle tracking based on feature similarity while maintaining memory efficiency.

Algorithm 1: The pseudo code of the single-camera tracking process.

```

Initialize buffer  $\leftarrow$  empty
Initialize max_buffer_size
Initialize threshold
WHILE new_frame_received DO:
    Extract features of all detected vehicles from the new frame
    IF buffer_size > 0 THEN
        Set all vehicle IDs in the new frame to 0
        Compute similarity matrix between buffer features and new frame features

        WHILE new vehicles remain unmatched DO
            Find the pair (buffer_vehicle, new_vehicle) with the highest similarity

            IF max_similarity > threshold THEN
                Assign buffer_vehicle ID to new_vehicle
                Set similarity_matrix[row=new_vehicle, col=buffer_vehicle]  $\leftarrow -\infty$ 
            ELSE
                Assign a new unique ID to new_vehicle
            END IF
        END WHILE
    ELSE
        Assign new IDs to all detected vehicles
    END IF
    Store all vehicle information (including features and IDs) in buffer

    IF buffer_size > max_buffer_size THEN
        Remove oldest records until buffer_size  $\leq$  max_buffer_size
    END IF
END WHILE

```

To maintain efficiency, older records are removed when the buffer exceeds its maximum size. If the buffer contains previous detections, a similarity matrix is computed between stored and newly detected objects. If the highest similarity score exceeds a pre-defined threshold, the object retains its existing ID; otherwise, a new ID is assigned. All detected objects and their IDs are stored for future comparisons, ensuring reliable tracking even at low frame rates.

We use cosine similarity for vehicle matching in both single- and cross-camera tracking. Each camera processes detections individually before matching vehicles across cameras. A threshold determines the similarity, while the buffer size controls how many past frames are stored. Each buffer entry contains detected vehicles and assigned IDs. Once the buffer reaches capacity, the oldest data are discarded.

For each new frame, detected vehicles are compared with those in the buffer using a ReID model to extract appearance features. If two vehicles exceed the similarity threshold, they are considered the same.

4.3.2. Cross-Camera Tracking Process

The pseudo code of a cross-camera is shown as Algorithm 2. This algorithm tracks groups of vehicles within a single camera view. It starts by initializing an empty matched set U (line 1). The system continuously processes incoming frames (line 3) and extracts all vehicle groups in the new frame, storing them in set Q (line 4). If no groups are detected in the frame, the algorithm skips to the next frame (lines 6–7).

If the matched set U is not empty (line 10), the algorithm computes a similarity matrix between the existing groups in U and the newly detected groups in Q (line 11). It then iteratively finds and matches the most similar pairs of groups from Q and U (lines 13–14). If the highest similarity score exceeds a predefined threshold (line 16), the group ID from U is assigned to all vehicles in the matched group q (line 17), and the similarity matrix entry for this pair is set to negative infinity to prevent duplicate assignments (line 18). If the similarity score is below the threshold (line 19), the algorithm merges successfully matched groups from U with similar groups in Q , updating Q accordingly (lines 20–21).

If no prior matched groups exist (line 24), all newly detected groups in Q are directly added to U (line 25). Finally, the matched set U is updated with the remaining unmatched groups from Q (line 28), and the process repeats for each new frame (line 29).

Algorithm 2: The pseudo code of the cross-camera tracking process.

```

Initialize matched set  $U \leftarrow$  empty
WHILE new_frame_received DO
    Extract all groups of vehicles under the new frame as set  $Q$ 
    IF  $Q$  is empty THEN
        CONTINUE
    END IF
    IF  $\text{size}(U) > 0$  THEN
        Compute similarity matrix between elements in  $U$  and elements in  $Q$ 
        WHILE  $Q$  is not empty DO
            Find  $(q, u)$  pair with the highest similarity from the matrix

            IF  $\text{max\_similarity} > \text{threshold}$  THEN
                Assign group ID of  $u$  to all vehicles in  $q$ 
                Set  $\text{similarity\_matrix}[q, u] \leftarrow -\infty$ 
            ELSE
                Merge successfully matched groups in  $U$  into corresponding similar groups
                in  $Q$ 
                Update  $Q \leftarrow Q'$ 
            END IF
        END WHILE
    ELSE
        Add all elements in  $Q$  to matched set:  $U \leftarrow Q$ 
    END IF
    Update matched set:  $U \leftarrow Q'$ 
END WHILE

```

For cross-camera pairing, vehicles with the same ID are first grouped within each camera. These groups are then matched across cameras to identify the same vehicle.

Initially, all groups from the first camera form a matched set. Vehicles from subsequent cameras are sequentially compared, and successfully matched groups are added to the set.

This process repeats for all cameras. A cross-camera threshold determines if two groups belong to the same vehicle.

To prevent incorrect pairings, restricted zones (e.g., parks, plazas) are defined where vehicles should not appear. Objects detected in these zones are ignored, reducing false positives. Figure 5 highlights these areas in red. These red-highlighted areas are manually marked to prevent incorrect object detection.



Figure 5. Example of region segmentation.

4.4. Experiments Results

4.4.1. Evaluation Metrics

In the context of Multi-Camera Multi-Object Tracking (MCMOT), a performance evaluation is essential to quantify the accuracy and robustness of tracking algorithms. Two widely used metrics for evaluating the tracking performance are IDF1 and MOTA. These metrics help assess the quality of object tracking, particularly in multi-camera environments where vehicles or objects may move between different cameras and need to be tracked across different views.

- IDF1 (Identification F1 Score) [37]

The IDF1 score is a metric used to assess the accuracy of identity preservation in object tracking tasks. It evaluates the harmonic mean of identity precision and identity recall, effectively measuring how well a tracker maintains consistent object identities over time. Unlike traditional precision–recall metrics, IDF1 explicitly accounts for identity switches (ID swaps)—instances where the tracker incorrectly reassigns an object’s identity during tracking.

A higher IDF1 score signifies a better tracking performance, as it indicates both accurate object associations (fewer mismatches between detected objects and their true identities) and greater identity consistency (fewer switches between different IDs). This makes IDF1 particularly useful for evaluating the long-term tracking accuracy in dynamic environments. Specifically, it is calculated as:

$$\text{IDF1} = \frac{2 \times \text{ID Precision} \times \text{ID Recall}}{\text{ID Precision} + \text{ID Recall}}$$

where *ID Precision* is the ratio of correctly matched object detections to all detections assigned to a specific identity and *ID Recall* is the ratio of correctly matched object detections to all ground truth objects for that identity.

This metric is crucial when evaluating MCMOT systems, as it emphasizes the importance of correctly identifying and maintaining the object's identity, which is challenging in multi-camera setups with occlusions and varying viewpoints.

- MOTA (Multiple Object Tracking Accuracy) [37]

The Multiple Object Tracking Accuracy (MOTA) score is a key metric used to evaluate the overall performance of an object tracking system. It provides a comprehensive assessment by considering errors related to object detection, tracking consistency, and identity preservation. MOTA is particularly useful for measuring the reliability of a tracker across an entire sequence.

MOTA takes into account three main types of errors:

1. False Positives (FP)—cases where the tracker incorrectly detects an object that does not exist.
2. False Negatives (FN)—cases where the tracker fails to detect a true object.
3. Identity Switches (ID Swaps)—cases where the tracker mistakenly reassigns an identity to a tracked object, disrupting tracking continuity.

MOTA is calculated as:

$$\text{MOTA} = 1 - \frac{\sum_i (FN_i + FP_i + IDSW_i)}{\sum_i (GT_i)}$$

where FN_i is the number of false negatives for the i -th object, FP_i is the number of false positives for the i -th object, $IDSW_i$ is the number of identity switches for the i -th object, and GT_i is the number of ground truth objects for the i -th frame.

A higher MOTA score indicates a better tracking performance, as it reflects fewer detection errors and more stable identity tracking over time. However, since MOTA does not consider the tracking precision directly (unlike IDF1), it is often used alongside IDF1 for a more complete evaluation of the tracking performance.

4.4.2. Evaluation Results

The loss curves for YOLOv9 and SwinReID are presented in Figures 6 and 7. As shown in Figure 6, the three losses for YOLOv9—box loss, class loss, and distribution focal loss—converge within five epochs. This rapid convergence can be attributed to the model being pre-trained on the Veri-776 dataset, which allows it to learn feature representations more efficiently. For SwinReID, Figure 7 illustrates the convergence behavior of its three losses—Triplet Loss, Center Loss, and Cross-Entropy Loss. From the figure, it can be observed that these losses gradually stabilize and reach convergence within 20 epochs, confirming the effectiveness of the training process in refining feature learning.

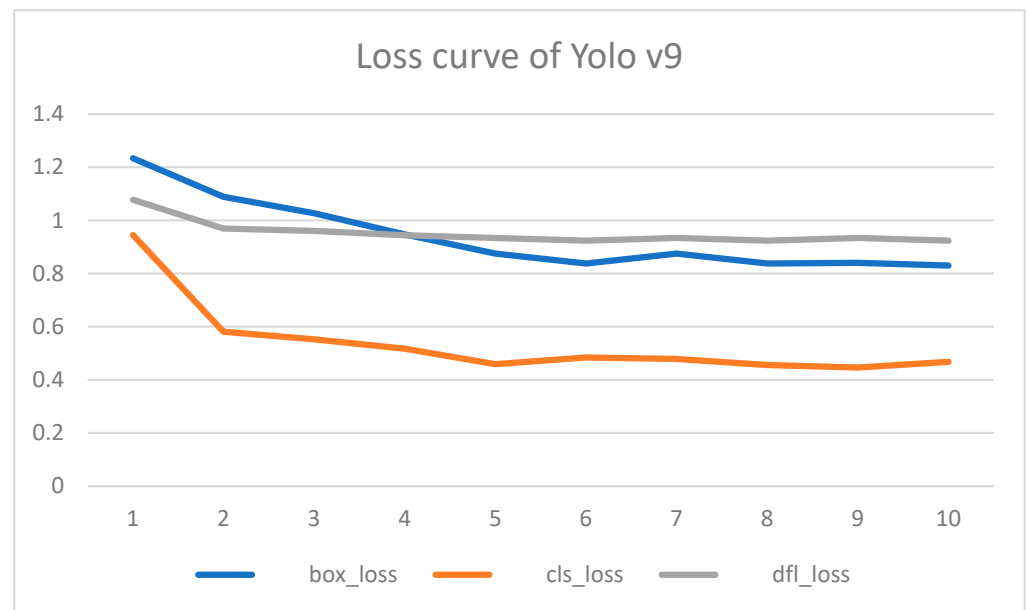


Figure 6. Loss curve of Yolo v9.

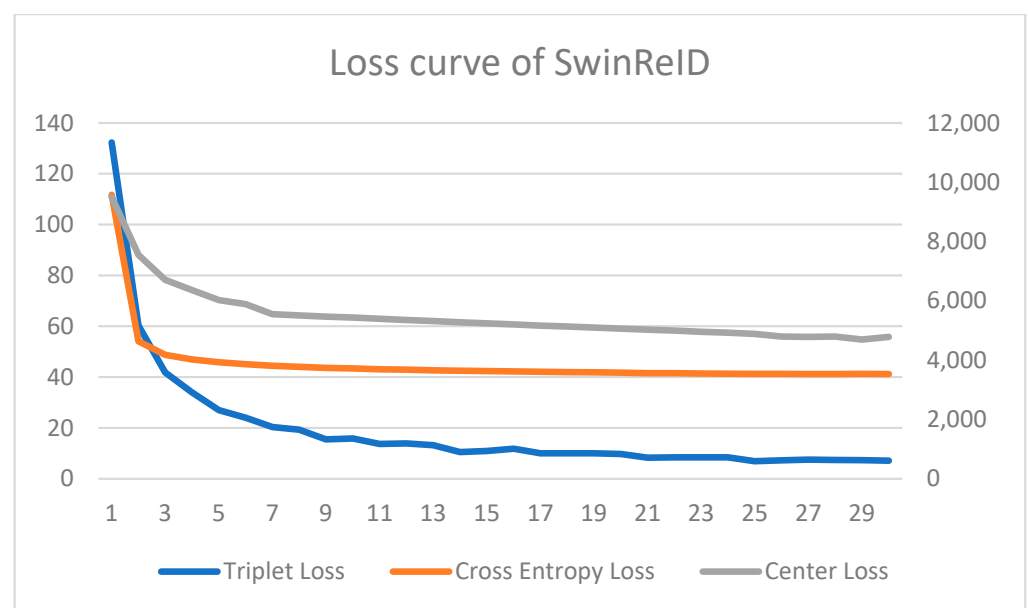


Figure 7. Loss curve of SwinReID.

The evaluation results of the multi-camera tracking system, based on the IDF1 and MOTA scores, show a strong overall performance across all cameras, as shown in Table 3. Camera 0 achieves the highest IDF1 score of 0.967, indicating excellent identity preservation, followed by Camera 1 (0.938) and Camera 3 (0.944), which also perform well in maintaining object identities. However, Camera 2, with an IDF1 score of 0.884, demonstrates some challenges in identity tracking, potentially due to factors like occlusions or misdetections. The average IDF1 score across all cameras is 0.940, reflecting a solid identity tracking performance with minor variations between the cameras.

Table 3. The results of our model in 8 cameras.

Camera	IDF1	MOTA
Camera 0	0.967	0.990
Camera 1	0.938	0.969
Camera 2	0.884	0.985
Camera 3	0.944	0.958
Camera 4	0.962	0.983
Camera 5	0.940	0.980
Camera 6	0.904	0.974
Camera 7	0.977	0.977
Average	0.940	0.977

In terms of MOTA, Camera 0 again leads with a score of 0.990, demonstrating high overall tracking accuracy with minimal errors. Camera 1 follows with a MOTA score of 0.969, and Camera 2 achieves a strong score of 0.985 despite its lower IDF1. The other cameras maintain MOTA scores ranging from 0.958 to 0.977, indicating a consistent performance in object detection and tracking. The average MOTA score of 0.977 suggests that the system performs well in reducing false positives, false negatives, and identity switches. Overall, the evaluation results indicate that the multi-camera tracking system excels in both identity preservation and object tracking accuracy, with only slight variations between individual cameras.

Additionally, we remove the second buffer to observe the changes in IDF1 and MOTA, and the result is shown in Table 4. Compared to the previous evaluation in Table 2, the performance across the cameras in terms of IDF1 and MOTA shows some variations. For IDF1, Camera 0 experiences a slight decrease, dropping from 0.967 to 0.921, while Camera 1 shows a noticeable reduction from 0.938 to 0.891. Camera 5 also experiences a decline in IDF1, from 0.944 to 0.892. These reductions indicate that the removal of the second buffer may have negatively impacted identity tracking, as fewer contextual clues are available across frames. In contrast, Camera 7 maintains a high IDF1 score of 0.977, which is consistent with its previous result, suggesting that removing the second buffer had a minimal impact on its tracking performance.

Table 4. The results of our model in 8 cameras w/o second buffer.

Camera	IDF1	MOTA
Camera 0	0.921	0.990
Camera 1	0.891	0.965
Camera 2	0.885	0.986
Camera 3	0.925	0.962
Camera 4	0.951	0.983
Camera 5	0.892	0.980
Camera 6	0.934	0.974
Camera 7	0.977	0.977
Average	0.922	0.977

As for MOTA, Camera 1 shows a slight decrease from 0.969 to 0.965, which, while noticeable, is not as significant as the drop in IDF1. Other cameras such as Camera 0, Camera 2, and Camera 4 maintain strong MOTA scores, with Camera 0 continuing to achieve 0.990 and Camera 2 at 0.986. Camera 3, however, shows a slight decrease from 0.962 to 0.962, indicating a minimal change in its performance. Camera 6 remains relatively stable with a MOTA of 0.974, showing only a minor reduction. Overall, the average MOTA across all cameras is 0.977, which is slightly lower than the 0.985 in Table 2. This suggests

that while most cameras performed similarly, the removal of the second buffer resulted in a small decline in the MOTA, particularly for Camera 1.

In conclusion, the removal of the second buffer led to noticeable declines in both the IDF1 and MOTA for some cameras, particularly Camera 1, while the overall tracking performance across most cameras remained strong. The results indicate that the second buffer plays a significant role in improving the tracking accuracy, especially for identity consistency and maintaining a low error rate in the system.

4.5. Discussions

This study presents a novel approach to Multi-Camera Multi-Object Tracking (MCMOT) in a low-FPS environment. While the proposed method demonstrates a strong performance in IDF1 and MOTA metrics, certain limitations must be acknowledged. Our approach primarily relies on appearance-based Re-ID, which makes it susceptible to identity mismatches, particularly in dense traffic conditions where heavy occlusion occurs. Additionally, although the buffer size is optimized for each camera, a dynamic buffer adaptation mechanism based on real-time conditions may further enhance the tracking performance. Another limitation is that the dataset primarily consists of fixed-angle surveillance cameras, which may limit the generalizability of the method. Future research could explore its effectiveness in pan-tilt-zoom (PTZ) camera settings or drone-based tracking.

To further advance MCMOT in low-FPS environments, future studies should explore how temporal constraints can be incorporated into the Re-ID model to enhance the tracking stability in low-FPS scenarios. Another promising direction is the use of reinforcement learning to dynamically optimize the buffer size rather than relying on fixed pre-optimized values. Additionally, a fusion of spatial and temporal data could be investigated to improve the tracking accuracy, particularly in multi-camera setups with large field-of-view variations. By addressing these questions, future research can further improve the robustness and adaptability of MCMOT systems in real-world applications.

5. Conclusions

Multi-Camera Multi-Object Tracking (MCMOT) is a critical task in deep learning, particularly in challenging environments with low frame rates. In this paper, we propose a solution to the MCMOT problem in low-FPS settings (below 1 FPS) by combining YOLOv9 for object detection and SwinReID for re-identification (ReID). YOLOv9 is utilized to detect objects within each frame, while SwinReID, inspired by the Swin Transformer model and TransReID, handles re-identification across different cameras.

The use of YOLOv9 enables accurate and efficient object detection under low-FPS conditions, allowing the system to identify vehicles in each frame. Meanwhile, SwinReID leverages the power of the Swin Transformer architecture, which has proven effective in image feature extraction tasks. By incorporating ReID, the system ensures that the same vehicles detected across multiple cameras can be correctly matched and tracked, even when they appear at different positions or times.

The experimental results demonstrate that the proposed method achieves a strong performance in both IDF1 and MOTA metrics, with an average IDF1 of 0.922 and an average MOTA of 0.977 across all cameras. These results validate the effectiveness of combining YOLOv9 and SwinReID for low-FPS MCMOT tasks. The model accurately tracks and re-identifies vehicles across different cameras, offering a reliable solution for real-world applications where low FPS is often a limiting factor.

Despite these promising results, there remains room for improvement. Due to FPS limitations in road surveillance images, our method currently relies solely on the vehicle appearance for the data association. Future research could explore network flow algorithms

to enhance the vehicle tracking and matching accuracy. Additionally, for cross-camera matching, techniques such as hierarchical clustering or DBSCAN could refine cluster determination. Many studies incorporate spatial and temporal constraints to mitigate mismatches caused by inconsistencies in time or location, and integrating such constraints could further improve the accuracy and robustness of multi-camera vehicle tracking.

Beyond YOLOv9 and the Swin Transformer, future work could investigate alternative models such as YOLOv5, YOLOv7, or simpler architectures in place of the Swin Transformer to evaluate potential performance trade-offs. Additionally, testing our model in high-FPS environments, such as MOT16 and MOT20, would allow for further performance analysis under varying conditions. Downsampling these datasets to simulate low-FPS scenarios could provide a more comprehensive evaluation of our model's adaptability across different frame rates. By addressing these aspects, future research can refine MCMOT techniques, improving their scalability and robustness for real-world applications.

Author Contributions: Conceptualization, Y.-H.H., C.-C.K., C.-H.L., K.-P.L., S.-Y.Y. and S.-M.Y.; methodology, Y.-H.H., C.-C.K., C.-H.L., K.-P.L., S.-Y.Y. and S.-M.Y.; software, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; validation, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; formal analysis, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; investigation, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; resources, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; data curation, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; writing—original draft preparation, Y.-H.H.; writing—review and editing, Y.-H.H.; visualization, Y.-H.H., C.-C.K., C.-H.L., K.-P.L. and S.-Y.Y.; supervision, S.-M.Y.; project administration, S.-M.Y.; funding acquisition, S.-Y.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Restrictions apply to the availability of these data. Data [28] were obtained from MOE AI competition and labeled data acquisition project and are available <https://www.aicup.tw/> (accessed on 19 March 2025) with the permission of the MOE AI competition and labeled data acquisition project. The DD100K MOT dataset was published as repository version 1.1.0, released on 2 December 2021 under the Apache License 2.0, where the copyright holder is the Visual Intelligence and Systems Group @ ETH Zurich.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

MCMOT	Multi-Camera Multi-Object Tracking
MOT	Multi-Object Tracking
ReID	Re-Identification
YOLO v9	You Only Look Once version9
MOTA	Multiple-Object Tracking Accuracy
IDF1	Identification F1 Score
Swin-ReID	Swin-Transformer Re-Identification.
DTC	Detection–tracking–counting

References

1. Tang, Z.; Naphade, M.; Liu, M.-Y.; Yang, X.; Birchfield, S.; Wang, S.; Kumar, R.; Anastasiu, D.; Hwang, J.-N. Cityflow: A city-scale benchmark for multi-target multi-camera vehicle tracking and re-identification. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 8797–8806.
2. Qian, Y.; Yu, L.; Liu, W.; Hauptmann, A.G. Electricity: An efficient multi-camera vehicle tracking system for intelligent city. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, Seattle, WA, USA, 14–19 June 2020; pp. 588–589.

3. Zhang, Y.; Li, X.; Wang, F.; Wei, B.; Li, L. A comprehensive review of one-stage networks for object detection. In Proceedings of the 2021 IEEE International Conference on Signal Processing, Communications and Computing (ICSPCC), Xi'an, China, 17–19 August 2021; IEEE: New York, NY, USA, 2021; pp. 1–6.
4. Luo, W.; Xing, J.; Milan, A.; Zhang, X.; Liu, W.; Kim, T.-K. Multiple object tracking: A literature review. *Artif. Intell.* **2021**, *293*, 103448.
5. Dai, Y.; Hu, Z.; Zhang, S.; Liu, L. A survey of detection-based video multi-object tracking. *Displays* **2022**, *75*, 102317. [[CrossRef](#)]
6. Naphade, M.; Wang, S.; Anastasiu, D.C.; Tang, Z.; Chang, M.-C.; Yao, Y. The 7th AI city challenge. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023; pp. 5538–5548.
7. Hassan, S.; Mujtaba, G.; Rajput, A.; Fatima, N. Multi-object tracking: A systematic literature review. *Multimed. Tools Appl.* **2024**, *83*, 43439–43492. [[CrossRef](#)]
8. Ha, S.V.-U.; Chung, N.M.; Nguyen, T.-C.; Phan, H.N. Tiny-pirate: A tiny model with parallelized intelligence for real-time analysis as a traffic counter. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 4119–4128.
9. Gloudemans, D.; Work, D.B. Fast vehicle turning-movement counting using localization-based tracking. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 4155–4164.
10. Kocur, V.; Ftacnik, M. Multi-class multi-movement vehicle counting based on CenterTrack. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 4009–4015.
11. Lu, J.; Xia, M.; Gao, X.; Yang, X.; Tao, T.; Meng, H. Robust and online vehicle counting at crowded intersections. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 4002–4008.
12. Tran, D.N.-N.; Pham, L.H.; Nguyen, H.-H.; Tran, T.H.-P.; Jeon, H.-J.; Jeon, J.W. A region-and-trajectory movement matching for multiple turn-counts at road intersection on edge device. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 4087–4094.
13. Tran, V.-H.; Dang, L.-H.-H.; Nguyen, C.-N.; Le, N.-H.-L.; Bui, K.-P.; Dam, L.-T. Real-time and robust system for counting movement-specific vehicle at crowded intersections. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Nashville, TN, USA, 19–25 June 2021; pp. 4228–4235.
14. Cheng, A.; Wang, Q.; Liu, F.; Li, X. DATrack: MCMOT Based on Feature Decoupling and Adaptive Motion Association. In Proceedings of the 2024 American Control Conference (ACC), Toronto, ON, Canada, 10–12 July 2024; IEEE: New York, NY, USA, 2024; pp. 4384–4389.
15. Wang, C.-Y.; Yeh, I.-H.; Liao, H.-Y.M. Yolov9: Learning what you want to learn using programmable gradient information. In Proceedings of the European Conference on Computer Vision, Milan, Italy, 29 September–2 October 2024; Springer: Berlin/Heidelberg, Germany, 2024; pp. 1–21.
16. Yang, B.; Huang, C.; Nevatia, R. Learning affinities and dependencies for multi-target tracking using a CRF model. In Proceedings of the CVPR 2011, Colorado Springs, CO, USA, 20–25 June 2011; IEEE: New York, NY, USA; pp. 1233–1240.
17. Pellegrini, S.; Ess, A.; Schindler, K.; Van Gool, L. You'll never walk alone: Modeling social behavior for multi-target tracking. In Proceedings of the 2009 IEEE 12th International Conference on Computer Vision, Kyoto, Japan, 29 September–2 October 2009; IEEE: New York, NY, USA, 2010; pp. 261–268.
18. Luo, W.; Kim, T.-K.; Stenger, B.; Zhao, X.; Cipolla, R. Bi-label propagation for generic multiple object tracking. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 23–28 June 2014; pp. 1290–1297.
19. Betke, M.; Hirsh, D.E.; Bagchi, A.; Hristov, N.I.; Makris, N.C.; Kunz, T.H. Tracking large variable numbers of objects in clutter. In Proceedings of the 2007 IEEE Conference on Computer Vision and Pattern Recognition, Minneapolis, MN, USA, 17–22 June 2007; IEEE: New York, NY, USA, 2007; pp. 1–8.
20. Lu, W.-L.; Ting, J.-A.; Little, J.J.; Murphy, K.P. Learning to track and identify players from broadcast sports videos. *IEEE Trans. Pattern Anal. Mach. Intell.* **2013**, *35*, 1704–1716. [[PubMed](#)]
21. Xing, J.; Ai, H.; Liu, L.; Lao, S. Multiple player tracking in sports video: A dual-mode two-way bayesian inference approach with progressive observation modeling. *IEEE Trans. Image Process.* **2010**, *20*, 1652–1667. [[CrossRef](#)] [[PubMed](#)]
22. Varghese, R.; Sambath, M. Yolov8: A novel object detection algorithm with enhanced performance and robustness. In Proceedings of the 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS), Chennai, India, 18–19 April 2024; IEEE: New York, NY, USA, 2024; pp. 1–6.
23. Wojke, N.; Bewley, A.; Paulus, D. Simple online and realtime tracking with a deep association metric. In Proceedings of the 2017 IEEE International Conference on Image Processing (ICIP), Beijing, China, 17–20 September 2017; IEEE: New York, NY, USA, 2018; pp. 3645–3649.
24. Yu, F.; Chen, H.; Wang, X.; Xian, W.; Chen, Y.; Liu, F.; Madhavan, V.; Darrell, T. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020; IEEE: New York, NY, USA, 2020; pp. 2633–2642.

25. Wang, Z.; Wang, L.; Shi, Z.; Zhang, M.; Geng, Q.; Jiang, N. A survey on person and vehicle re-identification. *IET Comput. Vis.* **2024**, *18*, 1235–1268.
26. Qian, Y.; Barthélemy, J.; Du, B.; Shen, J. Paying attention to vehicles: A systematic review on transformer-based vehicle re-identification. In *ACM Transactions on Multimedia Computing, Communications and Applications*; Association for Computing Machinery: New York, NY, USA, 2024.
27. Yi, X.; Wang, Q.; Liu, Q.; Rui, Y.; Ran, B. Advances in vehicle re-identification techniques: A survey. *Neurocomputing* **2025**, *614*, 128745.
28. Ministry of Education Artificial Intelligence Competition and Annotation Data Collection Project Office. MOE AI Competition and Labeled Data Acquisition Project. Available online: <https://tbrain.trendmicro.com.tw/Competitions/Details/33> (accessed on 29 January 2025).
29. Russakovsky, O.; Deng, J.; Su, H.; Krause, J.; Satheesh, S.; Ma, S.; Huang, Z.; Karpathy, A.; Khosla, A.; Bernstein, M.; et al. Imagenet large scale visual recognition challenge. *Int. J. Comput. Vis.* **2015**, *115*, 211–252. [[CrossRef](#)]
30. Zhong, Z.; Zheng, L.; Kang, G.; Li, S.; Yang, Y. Random erasing data augmentation. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; Volume 34, pp. 13001–13008.
31. Liu, Z.; Lin, Y.; Cao, Y.; Hu, H.; Wei, Y.; Zhang, Z.; Lin, S.; Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, QC, Canada, 10–17 October 2021; pp. 10012–10022.
32. He, S.; Luo, H.; Wang, P.; Wang, F.; Li, H.; Jiang, W. Transreid: Transformer-based object re-identification. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Montreal, QC, Canada, 11–17 October 2021; pp. 15013–15022.
33. Hoffer, E.; Ailon, N. Deep metric learning using triplet network. In *Similarity-Based Pattern Recognition, Proceedings of the Third International Workshop, SIMBAD 2015, Copenhagen, Denmark, 12–14 October 2015*; Proceedings 3; Springer: Berlin/Heidelberg, Germany, 2015; pp. 84–92.
34. Wen, Y.; Zhang, K.; Li, Z.; Qiao, Y. A discriminative feature learning approach for deep face recognition. In *Computer Vision—ECCV 2016, Proceedings of the 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016*; Proceedings, Part VII 14; Springer: Berlin/Heidelberg, Germany, 2016; pp. 499–515.
35. Loshchilov, I. Decoupled weight decay regularization. *arXiv* **2017**, arXiv:1711.05101.
36. Liu, X.; Liu, W.; Mei, T.; Ma, H. A deep learning-based approach to progressive vehicle re-identification for urban surveillance. In *Computer Vision—ECCV 2016, Proceedings of the 14th European Conference, Amsterdam, The Netherlands, 11–14 October 2016*; Proceedings, Part II 14; Springer: Berlin/Heidelberg, Germany, 2016; pp. 869–884.
37. Ristani, E.; Solera, F.; Zou, R.; Cucchiara, R.; Tomasi, C. Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2016; pp. 17–35.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.